

Cómo se entrena un VAE

ELBO, función de pérdida y truco de reparametrización

Maestría en Inteligencia Artificial

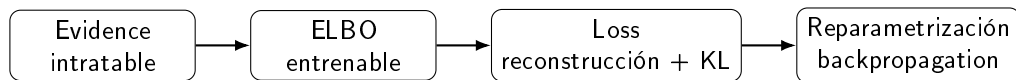
Universidad ORT Uruguay

29 de setiembre de 2026

- Formalizar el problema de máxima verosimilitud en un modelo latente.
- Derivar una cota entrenable para $\log p_{\theta}(x)$: la ELBO.
- Interpretar los dos términos de la loss de un VAE.
- Entender por qué muestrear z dificulta el gradiente del encoder.
- Derivar el truco de reparametrización para una gaussiana diagonal.

Hoy pasamos de la arquitectura al algoritmo de entrenamiento.

El recorrido de hoy



Cada herramienta aparecerá como respuesta a un problema concreto.

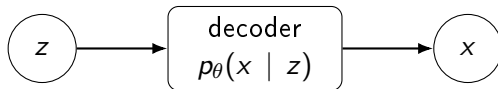
Repaso: el modelo generativo

En un VAE proponemos el siguiente proceso:

$$z \sim p(z), \quad x \sim p_{\theta}(x \mid z)$$

Por lo tanto, la conjunta factoriza como:

$$p_{\theta}(x, z) = p(z)p_{\theta}(x \mid z)$$



La likelihood que queremos maximizar

El dataset solo contiene observaciones:

$$\mathcal{T} = \{x^{(1)}, \dots, x^{(m)}\}$$

No observamos qué z produjo cada dato. Debemos marginalizarlo:

$$p_{\theta}(x) = \int p_{\theta}(x, z) dz = \int p_{\theta}(x \mid z) p(z) dz$$

Objetivo

Queremos maximizar $\sum_{x \in \mathcal{T}} \log p_{\theta}(x)$, pero evaluar cada término exige una integral.

Una mezcla con infinitos componentes

Si z fuera discreto:

$$p_{\theta}(x) = \sum_z p_{\theta}(x \mid z)p(z)$$

Con z continuo:

$$p_{\theta}(x) = \int p_{\theta}(x \mid z)p(z) dz$$

Cada valor de z selecciona una distribución del decoder. La marginal combina un continuo de esas distribuciones.

El modelo puede ser fácil de muestrear y difícil de evaluar.

Por qué la integral suele ser intratable

Aunque el prior sea simple,

$$p(z) = \mathcal{N}(0, I),$$

el decoder es una transformación neuronal no lineal dentro de:

$$\int p_{\theta}(x \mid z) p(z) dz.$$

En general:

- no existe una solución analítica;
- integrar numéricamente en alta dimensión es costoso;
- tampoco podemos evaluar fácilmente el posterior exacto.

El posterior exacto también es difícil

Por Bayes:

$$p_{\theta}(z \mid x) = \frac{p_{\theta}(x \mid z)p(z)}{p_{\theta}(x)}$$

El denominador es justamente la integral intratable.

Introducimos entonces un modelo de inferencia:

$$q_{\phi}(z \mid x) \approx p_{\theta}(z \mid x)$$

El encoder amortiza la inferencia: una red aprende a inferir z para cualquier x .

Una identidad útil

Para cualquier distribución $q_\phi(z \mid x)$:

$$\log p_\theta(x) = \mathcal{L}_{\text{ELBO}}(x; \theta, \phi) + \text{KL}(q_\phi(z \mid x) \parallel p_\theta(z \mid x))$$

Como la KL nunca es negativa:

$$\mathcal{L}_{\text{ELBO}}(x; \theta, \phi) \leq \log p_\theta(x)$$

Idea

Si la evidence no puede calcularse, maximizamos una cota inferior que sí podemos estimar.

Derivación I: escribir una constante como esperanza

Para un x fijo, $\log p_{\theta}(x)$ no depende de z :

$$\log p_{\theta}(x) = \mathbb{E}_{z \sim q_{\phi}(z|x)} [\log p_{\theta}(x)]$$

Por Bayes:

$$p_{\theta}(x) = \frac{p_{\theta}(x, z)}{p_{\theta}(z | x)}$$

Entonces:

$$\log p_{\theta}(x) = \mathbb{E}_{q_{\phi}} \left[\log \frac{p_{\theta}(x, z)}{p_{\theta}(z | x)} \right].$$

Derivación II: multiplicar y dividir por q_ϕ

$$\log p_\theta(x) = \mathbb{E}_{q_\phi} \left[\log \left(\frac{p_\theta(x, z)}{q_\phi(z | x)} \frac{q_\phi(z | x)}{p_\theta(z | x)} \right) \right]$$

Separamos el logaritmo:

$$\log p_\theta(x) = \underbrace{\mathbb{E}_{q_\phi} \left[\log \frac{p_\theta(x, z)}{q_\phi(z | x)} \right]}_{\mathcal{L}_{\text{ELBO}}(x; \theta, \phi)} + \underbrace{\mathbb{E}_{q_\phi} \left[\log \frac{q_\phi(z | x)}{p_\theta(z | x)} \right]}_{\text{KL}(q_\phi \| p_\theta)}$$

Qué hace más ajustada a la cota

$$\log p_{\theta}(x) - \mathcal{L}_{\text{ELBO}}(x; \theta, \phi) = \text{KL}(q_{\phi}(z \mid x) \parallel p_{\theta}(z \mid x))$$

- Si q_{ϕ} está lejos del posterior, la cota es floja.
- Si $q_{\phi} = p_{\theta}(z \mid x)$, la KL vale cero.
- En ese caso, la ELBO coincide con $\log p_{\theta}(x)$.

Optimizar ϕ mejora la aproximación inferencial; optimizar θ mejora el modelo generativo.

La forma práctica de la ELBO

Usamos:

$$p_{\theta}(x, z) = p_{\theta}(x \mid z)p(z)$$

Entonces:

$$\mathcal{L}_{\text{ELBO}}(x) = \mathbb{E}_{q_{\phi}(z \mid x)} [\log p_{\theta}(x \mid z)] - \text{KL}(q_{\phi}(z \mid x) \parallel p(z))$$

Esta es la expresión central para entrenar un VAE.

De maximizar ELBO a minimizar una loss

Los optimizadores suelen minimizar. Definimos:

$$\mathcal{L}_{\text{VAE}}(x) = -\mathcal{L}_{\text{ELBO}}(x)$$

$$\mathcal{L}_{\text{VAE}}(x) = \underbrace{-\mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x | z)]}_{\text{reconstrucción / NLL}} + \underbrace{\text{KL}(q_{\phi}(z | x) \| p(z))}_{\text{regularización latente}}$$

No son dos términos elegidos ad hoc: ambos salen de la cota variacional.

$$-\mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x | z)]$$

Pregunta que responde:

¿El z inferido permite al decoder asignar alta probabilidad al dato x ?

No exige que $\hat{x}$ sea idéntico a x en un sentido genérico. La forma concreta depende de la distribución elegida para $p_{\theta}(x | z)$.

Si la likelihood es Bernoulli

Para datos binarios:

$$p_{\theta}(x \mid z) = \prod_{j=1}^d \pi_j(z)^{x_j} (1 - \pi_j(z))^{1-x_j}$$

La negative log-likelihood es:

$$-\log p_{\theta}(x \mid z) = \sum_{j=1}^d [-x_j \log \pi_j(z) - (1 - x_j) \log(1 - \pi_j(z))]$$

Reconstrucción = binary cross-entropy.

Si la likelihood es gaussiana

Si:

$$p_{\theta}(x \mid z) = \mathcal{N}(x; f_{\theta}(z), \sigma_x^2 I)$$

entonces:

$$-\log p_{\theta}(x \mid z) = \frac{1}{2\sigma_x^2} \|x - f_{\theta}(z)\|^2 + C$$

Con varianza fija, minimizar la NLL equivale —salvo escala y constante— a minimizar MSE.

Regla

BCE o MSE no se eligen por costumbre: se derivan de $p_{\theta}(x \mid z)$.

$$\text{KL}(q_{\phi}(z \mid x) \parallel p(z))$$

Compara la distribución que el encoder asigna a cada dato con el prior.

- Evita códigos aislados y arbitrarios.
- Favorece una región latente que pueda recorrerse y muestrearse.
- Hace compatibles entrenamiento y generación desde $z \sim p(z)$.

No obliga a cada z muestreado a ser cero: penaliza distribuciones completas.

Encoder gaussiano diagonal

Usualmente:

$$q_{\phi}(z \mid x) = \mathcal{N}(z; \mu_{\phi}(x), \text{diag}(\sigma_{\phi}^2(x)))$$

y elegimos:

$$p(z) = \mathcal{N}(0, I).$$

El encoder produce dos vectores:

$$\mu_{\phi}(x) \in \mathbb{R}^k, \quad \sigma_{\phi}(x) \in \mathbb{R}_{>0}^k.$$

Para el encoder diagonal y el prior estándar:

$$\text{KL}(q_\phi(z \mid x) \parallel p(z)) = \frac{1}{2} \sum_{j=1}^k (\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1)$$

No necesitamos Monte Carlo para este término.

- μ_j^2 penaliza medias lejos de cero.
- $\sigma_j^2 - \log \sigma_j^2 - 1$ se minimiza en $\sigma_j^2 = 1$.

La tensión que aprende el VAE

Reconstrucción

$$-\mathbb{E}_{q_\phi} \log p_\theta(x \mid z)$$

- conservar información de x ;
- distinguir datos;
- reconstruir bien.

KL

$$\text{KL}(q_\phi(z \mid x) \parallel p(z))$$

- acercarse al prior;
- ocupar un espacio regular;
- poder generar.

Un VAE aprende un compromiso entre información y estructura.

Cuidado con las escalas

La reconstrucción suele sumar sobre píxeles o features:

$$\mathcal{L}_{\text{rec}} = \sum_{j=1}^d \ell_j.$$

La KL suma sobre dimensiones latentes:

$$\mathcal{L}_{\text{KL}} = \sum_{j=1}^k \text{KL}_j.$$

Si una implementación promedia un término y suma el otro, cambia el balance. Antes de comparar losses, hay que revisar las reducciones: `sum` vs `mean`.

La esperanza se estima con Monte Carlo

El término de reconstrucción contiene:

$$\mathbb{E}_{z \sim q_{\phi}(z | x)} [\log p_{\theta}(x | z)].$$

Lo aproximamos con K muestras:

$$\mathbb{E}_{q_{\phi}}[f(z)] \approx \frac{1}{K} \sum_{k=1}^K f(z^{(k)}), \quad z^{(k)} \sim q_{\phi}(z | x).$$

En VAE suele usarse $K = 1$ por dato; el minibatch aporta más muestras.

Para el decoder, el gradiente es directo

Una vez muestreado z , podemos calcular:

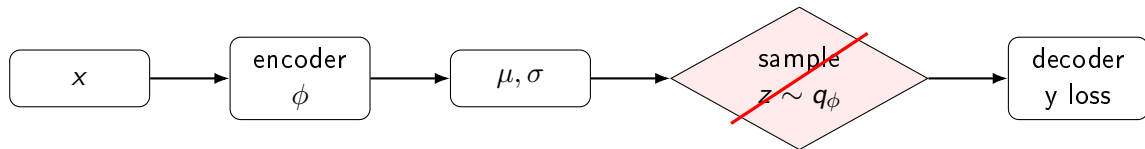
$$\nabla_{\theta} \log p_{\theta}(x \mid z)$$

El decoder es una función diferenciable de θ :

$$z \longrightarrow f_{\theta}(z) \longrightarrow \log p_{\theta}(x \mid z).$$

El problema aparece con ϕ : la distribución de la que salió z depende de los parámetros del encoder.

El nodo de muestreo corta el camino ingenuo



Una muestra aleatoria no viene con una derivada ordinaria que indique cómo cambiaría al mover μ_ϕ o σ_ϕ .

Queremos:

$$\nabla_{\phi} \mathbb{E}_{z \sim q_{\phi}(z | x)} [f(z)] .$$

Pero ϕ aparece en la distribución:

$$\nabla_{\phi} \int q_{\phi}(z | x) f(z) dz .$$

No alcanza con muestrear z y derivar solamente $f(z)$: necesitamos capturar cómo cambia la muestra cuando cambia ϕ .

Separar ruido y parámetros

En lugar de muestrear directamente:

$$z \sim \mathcal{N}(\mu_{\phi}(x), \text{diag}(\sigma_{\phi}^2(x))) ,$$

muestreemos ruido estándar:

$$\epsilon \sim \mathcal{N}(0, I),$$

y construimos:

$$z = \mu_{\phi}(x) + \sigma_{\phi}(x) \odot \epsilon$$

La distribución de z es la misma, pero ahora el azar no depende de ϕ .

Por qué produce la distribución correcta

En una dimensión:

$$\epsilon \sim \mathcal{N}(0, 1), \quad z = \mu + \sigma\epsilon.$$

Entonces:

$$\begin{aligned}\mathbb{E}[z] &= \mu + \sigma\mathbb{E}[\epsilon] = \mu \\ \text{Var}(z) &= \sigma^2 \text{Var}(\epsilon) = \sigma^2.\end{aligned}$$

Por lo tanto:

$$z \sim \mathcal{N}(\mu, \sigma^2).$$

La esperanza reparametrizada

Antes:

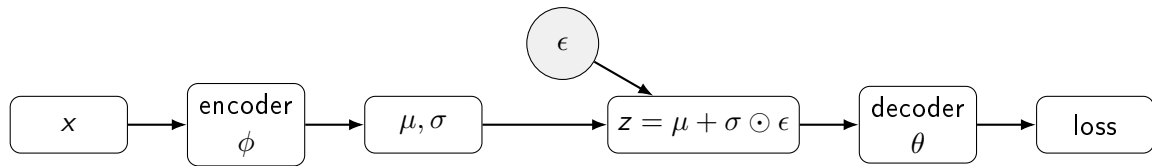
$$\mathbb{E}_{z \sim q_{\phi}(z|x)}[f(z)].$$

Después:

$$\mathbb{E}_{\epsilon \sim \mathcal{N}(0,I)} [f(\mu_{\phi}(x) + \sigma_{\phi}(x) \odot \epsilon)].$$

Ahora la distribución de muestreo es fija y ϕ aparece dentro de una función diferenciable.

Ahora sí hay un camino para backpropagation



Condicionado al ruido sorteado, toda la ruta desde la loss hasta ϕ es diferenciable.

El estimador de gradiente

$$\nabla_{\phi} \mathbb{E}_{\epsilon \sim \mathcal{N}(0, I)} [f(\mu_{\phi} + \sigma_{\phi} \odot \epsilon)]$$

Como la distribución de ϵ no depende de ϕ :

$$= \mathbb{E}_{\epsilon} [\nabla_{\phi} f(\mu_{\phi} + \sigma_{\phi} \odot \epsilon)].$$

Con Monte Carlo:

$$\approx \frac{1}{K} \sum_{k=1}^K \nabla_{\phi} f(\mu_{\phi} + \sigma_{\phi} \odot \epsilon^{(k)}).$$

Implementación estable: predecir log-varianza

Una red no debería emitir σ directamente: podría producir valores negativos.

En práctica el encoder devuelve:

$$\mu_{\phi}(x), \quad \log \sigma_{\phi}^2(x) = \text{logvar}.$$

Luego:

$$\sigma_{\phi}(x) = \exp\left(\frac{1}{2}\text{logvar}\right)$$
$$z = \mu_{\phi}(x) + \exp\left(\frac{1}{2}\text{logvar}\right) \odot \epsilon.$$

Loss para un minibatch

Para un minibatch B :

$$\mathcal{L}_{\text{VAE}}(B) = \frac{1}{|B|} \sum_{x \in B} \left[-\log p_{\theta}(x \mid z) + \frac{1}{2} \sum_{j=1}^k (\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1) \right]$$

$$\epsilon \sim \mathcal{N}(0, I), \quad z = \mu_{\phi}(x) + \sigma_{\phi}(x) \odot \epsilon.$$

La reconstrucción está estimada con una muestra; la KL se calcula en forma cerrada.

Un paso de entrenamiento

Para cada minibatch x :

1. Encoder: calcular μ y $\log\text{var}$.
2. Ruido: muestrear $\epsilon \sim \mathcal{N}(0, I)$.
3. Reparametrizar:

$$z = \mu + \exp(0,5 \log\text{var}) \odot \epsilon.$$

4. Decoder: obtener los parámetros de $p_{\theta}(x \mid z)$.
5. Calcular reconstrucción + KL.
6. Backpropagar y actualizar θ, ϕ .

Entrenar, reconstruir y generar

Entrenar

$$x \rightarrow q_{\phi}(z \mid x)$$

$$z \rightarrow p_{\theta}(x \mid z)$$

Usa ambas redes y ambas partes de la loss.

Reconstruir

$$x \rightarrow q_{\phi}(z \mid x)$$

$$z \rightarrow \hat{x}$$

Evalúa el circuito encoder-decoder.

Generar

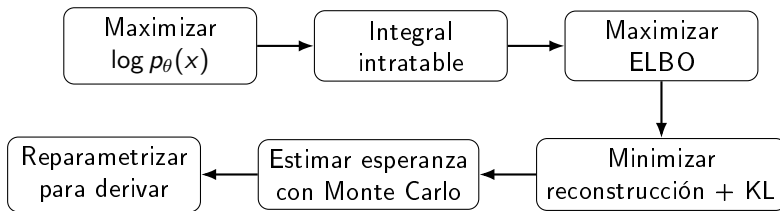
$$z \sim p(z)$$

$$z \rightarrow x \sim p_{\theta}(x \mid z)$$

No necesita el encoder.

- **“La reconstruction loss siempre es MSE.”**
No: depende de la likelihood elegida.
- **“La KL compara la reconstrucción con el dato.”**
No: compara $q_\phi(z \mid x)$ con el prior $p(z)$.
- **“La ELBO es la likelihood exacta.”**
Solo coincide si q_ϕ iguala al posterior exacto.
- **“Reparametrizar elimina el azar.”**
No: traslada el azar a ϵ , cuya distribución no depende de ϕ .

El mapa completo



- La likelihood marginal de un modelo latente exige integrar z .
- El encoder $q_{\phi}(z \mid x)$ aproxima el posterior intratable.
- La ELBO es una cota inferior de $\log p_{\theta}(x)$.
- La loss negativa combina NLL de reconstrucción y KL al prior.
- La reparametrización separa el ruido de los parámetros del encoder.
- Así podemos entrenar encoder y decoder juntos con backpropagation.

Idea central

La ELBO resuelve qué optimizar; la reparametrización resuelve cómo propagar el gradiente a través de una muestra.

$$\mathcal{L}_{\text{VAE}}(x) = -\mathbb{E}_{q_{\phi}(z|x)} \log p_{\theta}(x | z) + \text{KL}(q_{\phi}(z | x) \| p(z))$$
$$z = \mu_{\phi}(x) + \sigma_{\phi}(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

Preguntas para chequear

1. ¿Por qué no podemos maximizar directamente $\log p_{\theta}(x)$?
2. ¿Qué diferencia hay entre el posterior exacto y $q_{\phi}(z \mid x)$?
3. ¿Por qué la ELBO es una cota inferior?
4. ¿Qué determina si la reconstrucción usa BCE o MSE?
5. ¿Qué ocurriría si quitáramos el término KL?
6. ¿Qué cambia matemáticamente al reparametrizar z ?

- Diederik P. Kingma y Max Welling. *Auto-Encoding Variational Bayes*. ICLR, 2014.
- Carl Doersch. *Tutorial on Variational Autoencoders*. arXiv, 2016/2021.
- Danilo J. Rezende, Shakir Mohamed y Daan Wierstra. *Stochastic Backpropagation and Approximate Inference in Deep Generative Models*. ICML, 2014.
- Kevin P. Murphy. *Probabilistic Machine Learning: Advanced Topics*. MIT Press, 2023.
- Jakub M. Tomczak. *Deep Generative Modeling*. Springer, 2022.